I am completing Scenario 18 — Blackjack vs Dealer for my Software Engineering Vibe Coding lab.
I have fixed and tested the Ace-scoring function in cards.py. Do not change that function.
Now help me complete Task 2 in game.py.
Requirements:

1. Handle natural blackjack correctly.
2. Handle player bust and dealer bust correctly.
3. Make the dealer draw while its hand value is below 17.
4. Handle pushes (ties) correctly.
5. Prevent player actions after a round has ended.
6. Preserve the existing project structure and terminal-based gameplay.
7. Do not implement the wager mechanism yet; that is Task 3.
8. Explain each change and give me test cases with expected results.
9. Do not replace the entire project or add unnecessary dependencies.

First inspect the existing code and explain your proposed changes. Then provide the updated game.py. from cards import Deck, hand_value

```
class Blackjack:
    def __init__(self):
        self.chips = 100

    def show(self, player, dealer, hide=True):
        shown_dealer = ["??"] if hide else [f"{r}{s}" for r, s in dealer]
        print("Dealer:", " ".join(shown_dealer))
        print("Player:", " ".join(f"{r}{s}" for r, s in player),
              "=", hand_value(player))

    def round(self):
        deck = Deck()
        player = [deck.draw(), deck.draw()]
        dealer = [deck.draw(), deck.draw()]
        self.show(player, dealer)
```

```
            while hand_value(player) < 21:
                key = input("[h]it [s]tand [q]uit: ").strip().lower()
                if key == "q":
                    return False
                if key == "s":
                    break
                if key == "h":
                    player.append(deck.draw())
                    self.show(player, dealer)
                    if hand_value(player) > 21:
                        print("Bust.")
                        return True
            while hand_value(dealer) < 17:
                dealer.append(deck.draw())

            self.show(player, dealer, hide=False)
            pv, dv = hand_value(player), hand_value(dealer)
            if dv > 21 or pv > dv:
                self.chips += 10
                print("Player wins.")
            elif pv < dv:
                self.chips -= 10
                print("Dealer wins.")
            else:
                print("Push.")
            return True

        def run(self):
            print("Blackjack — starting chips:", self.chips)
            while self.chips > 0:
                if not self.round():
                    return
                if input("Play again? [y/n]: ").strip().lower() != "y":
                    return
game.py     import random

RANKS = ["2", "3", "4", "5", "6", "7", "8", "9", "10", "J", "Q", "K", "A"]
SUITS = ["C", "D", "H", "S"]
```

```
class Deck:
    def __init__(self):
        self.cards = [(rank, suit) for suit in SUITS for rank in RANKS]
        random.shuffle(self.cards)

    def draw(self):
        return self.cards.pop() if self.cards else None
```

```
def hand_value(hand):
    value = 0
    aces = 0

    for rank, _ in hand:
        if rank == "A":
            value += 11
            aces += 1
        elif rank in {"J", "Q", "K"}:
            value += 10
        else:
            value += int(rank)

    while value > 21 and aces > 0:
        value -= 10
        aces -= 1

    return value
```

cards.py

👍 Your `cards.py` Ace-scoring function is already handling the important rule: an Ace counts as 11 when possible and drops to 1 when the hand would otherwise exceed 21. **We will leave** `cards.py` **unchanged** and make targeted changes only to `game.py`.

1. Inspecting your existing `game.py`

Your current implementation already has the correct basic structure:

- `Blackjack` class manages the game.
- `show()` displays the player and dealer hands.
- `round()` handles a single round.
- `run()` lets the player start another round.
- `Deck` and `hand_value()` are imported from `cards.py`.

The main problems are in the round's logic.

Requirement	Current issue	Proposed fix
Natural blackjack	A starting total of 21 skips player input but doesn't get special treatment.	Check both initial hands for blackjack before normal play.
Player bust	Busting immediately returns, before revealing the dealer's hand.	Reveal the dealer's hand and finish the round without playing the dealer's turn.
Dealer bust	The existing comparison handles this, but only as part of general winner selection.	Explicitly determine the outcome after the dealer finishes drawing.
Dealer draws below 17	Already implemented correctly.	Keep the existing `while hand_value(dealer) < 17` rule.
Pushes	Ties are handled, but the outcome logic is less explicit.	Check for equal totals and report a push.
No actions after the round ends	Some outcomes return early; natural blackjack isn't explicitly resolved.	Resolve terminal outcomes before prompting and return from the round.
Wagers	The code changes chips by 10 on wins and losses.	Remove these chip changes for Task 2; leave wager handling for Task 3.

One important distinction: **a natural blackjack means 21 with the initial two cards**, not a total of 21 reached after hitting.

2. Updated `game.py`

Replace the contents of your existing `game.py` with the code below. It preserves your class structure, terminal gameplay, and `cards.py` import.

‹/› Python

```python
from cards import Deck, hand_value


class Blackjack:
    def __init__(self):
        self.chips = 100
```

```
    def show(self, player, dealer, hide=True):
        shown_dealer = (
            ["??"] if hide
            else [f"{r}{s}" for r, s in dealer]
        )
        print("Dealer:", " ".join(shown_dealer))
        print(
            "Player:", " ".join(f"{r}{s}" for r, s in player),
            "=", hand_value(player)
        )

    def round(self):
```

3. Explanation of each change

Change 1 — Natural blackjack

Python

```
player_natural = pv == 21 and len(player) == 2
dealer_natural = dv == 21 and len(dealer) == 2
```

These checks happen immediately after the initial hands are dealt.

- Both players have natural blackjack → push.
- Only the player has natural blackjack → player wins.
- Only the dealer has natural blackjack → dealer wins.

The `return True` ends the round immediately, so the player cannot hit or stand after a natural blackjack.

Change 2 — Player bust

Python

```
if hand_value(player) > 21:
    print("Bust. Player loses.")
    self.show(player, dealer, hide=False)
    return True
```

Once the player exceeds 21, the round ends immediately. The dealer's hidden card is revealed, but the dealer does not draw any more cards because the player has already lost.

Change 3 — Dealer draws until 17

```python
while hand_value(dealer) < 17:
    dealer.append(deck.draw())
```

This is already correct in your original implementation, so it remains unchanged. The dealer keeps drawing while the hand is worth 16 or less, including a soft 16.

Change 4 — Resolve dealer busts, wins and pushes

The outcome checks are now explicit:

1. Dealer exceeds 21 → player wins.
2. Player's total is higher → player wins.
3. Dealer's total is higher → dealer wins.
4. Equal totals → push.

The dealer's bust is checked first so that a busted dealer cannot win simply because their final total is otherwise greater than the player's.

Change 5 — Remove chip changes for Task 2

Your original code contained:

```python
self.chips += 10
self.chips -= 10
```

Both are removed. Task 2 should establish the game rules without implementing the wager mechanism, which belongs to Task 3. The `chips` variable and initial value of `100` are preserved.

4. Test cases and expected results

Use these scenarios to verify the behavior. The hands below are illustrative test setups; your normal shuffled deck won't necessarily produce them in order.

Test	Player hand	Dealer hand	Expected result

1. Player natural	A♠, K♥	9♦, 7♣	Player wins with blackjack; no prompt
2. Dealer natural	10♠, 8♥	A♦, K♣	Dealer wins with blackjack; no prompt
3. Both natural	A♠, K♥	A♦, Q♣	Push; no prompt
4. Player bust	10♠, 8♥, 5♦	10♣, 7♦	Player loses immediately; dealer hand revealed
5. Dealer bust	10♠, 9♥	10♣, 6♦, 8♥	Dealer busts at 24; player wins
6. Player wins	10♠, 9♥	10♣, 8♦	Player wins, 19 vs. 18
7. Dealer wins	10♠, 7♥	10♣, 9♦	Dealer wins, 17 vs. 19
8. Push	10♠, 8♥	K♣, 8♦	Push, 18 vs. 18
9. Dealer stands on 17	10♠, 9♥	10♣, 7♦	Dealer stops at 17; player wins
10. Hit to 21	10♠, 6♥, 5♦	10♣, 7♦	Player reaches 21; dealer plays, then outcome is resolved

For Test 7, the correct result is **dealer wins, 17 vs. 19** only if the totals are interpreted accordingly; the listed hands actually total 17 vs. 19, so the expected winner is the player. To test a dealer win, use player 17 versus dealer 19 instead, for example player `10♠, 7♥` and dealer `10♣, 9♦` — 17 vs. 19, so the dealer wins.

5. How to run and verify it

From your project directory in Git Bash:

Bash

```
python game.py
```

If your project has a separate entry-point file that creates the `Blackjack` instance, run that file instead.

Verify these three things first:

- Natural blackjack ends the round without showing the action prompt.
- Busting ends the round without allowing another hit.
- Your chips remain at `100` regardless of the outcome.

Task 2 completion check: the game handles natural blackjack, busts, dealer drawing, pushes, and round termination without implementing wagers. Your existing `cards.py` remains untouched.

```
PS C:\Users\manis\Desktop\18_blackjack> python game.py
PS C:\Users\manis\Desktop\18_blackjack>
```

That's actually expected, Manish. 👍 Your `game.py` defines the `Blackjack` class, but it never creates an instance or calls `run()`. So Python finishes executing the file without displaying anything.

Fix: Add 2 lines at the bottom of `game.py`

Keep all your existing code and add this at the very end:

Python

```python
if __name__ == "__main__":
    Blackjack().run()
```

Now run it again

In PowerShell:

powershell

```powershell
python game.py
```

You should see output similar to:

Plain text

```
Blackjack — starting chips: 100
Dealer: ??
Player: 8H 7S = 15
[h]it [s]tand [q]uit:
```

Your cards and totals will vary because the deck is shuffled.

Why this works: `Blackjack().run()` creates a game instance and starts the game loop. The `if __name__ == "__main__":` condition ensures this happens when you run `game.py` directly, but not when another file imports it.

Once it starts, test `h` (hit), `s` (stand), and `q` (quit). Remember that natural blackjack and bust outcomes should end the round without another player-action prompt.

```
PS C:\Users\manis\Desktop\18_blackjack> python game.py
PS C:\Users\manis\Desktop\18_blackjack> python game.py
Blackjack — starting chips: 100
Dealer: ??
Player: AC 3D = 14
[h]it [s]tand [q]uit:
```

Perfect! ✅ Your Blackjack game is now running correctly.

Your output:

```
Plain text

Blackjack — starting chips: 100
Dealer: ??
Player: AC 3D = 14
[h]it [s]tand [q]uit:
```

Here's what to do now:

- **Type** `h` — Hit: draw another card.
- **Type** `s` — Stand: stop drawing and let the dealer play.
- **Type** `q` — Quit: exit the current round.

Test these behaviors

1. **Hit:** Enter `h` and verify that a new card appears and the player total updates.
2. **Stand:** Start another round, enter `s`, and verify that the dealer reveals its cards and draws until its total is at least 17.
3. **Quit:** Enter `q` and verify that the game exits.
4. **Natural blackjack:** When the initial two cards total 21, verify that the round ends without asking for a hit or stand.
5. **Chips:** Verify that the starting chips remain at 100 after a completed round.

One note: `AC` represents Ace of Clubs. Your displayed total of 14 is consistent with an Ace counting as 11 plus a 3.

Next step: Test `h` and `s` now. If both work, we can verify the edge cases and confirm Task 2 is ready to submit.

```
PS C:\Users\manis\Desktop\18_blackjack> python game.py
Blackjack — starting chips: 100
Dealer: ??
Player: AC 3D = 14
[h]it [s]tand [q]uit: h
Dealer: ??
Player: AC 3D 10D = 14
[h]it [s]tand [q]uit: s
Dealer: KC 9S
Player: AC 3D 10D = 14
Dealer wins.
Play again? [y/n]:
```

Great! ✅ Your **Hit and Stand functionality is working**, and the round ended correctly.

Let's verify your output:

- **Initial hand:** `AC 3D` — Ace + 3 = 14. Correct.
- **After hitting:** `AC 3D 10D` — the displayed total remains 14. Correct, because the Ace changes from 11 to 1 when necessary? In this particular hand, Ace + 3 + 10 totals 24 if the Ace stays 11, so it drops to 1, giving **14**. Correct.
- **After standing:** Dealer shows `KC 9S` — 10 + 9 = 19.
- **Result:** Dealer wins. Correct, because 19 beats 14.
- **Play again prompt:** Correct. The round ended and the game is ready for another round.

Next, test two things

1. Type `n` at `Play again? [y/n]:` to exit.
2. Run `python game.py` again and test `q` at the action prompt. It should quit without showing the play-again prompt.

Then we should verify the special cases: natural blackjack, player bust, dealer bust, and pushes. Because your deck is shuffled, those cases won't necessarily appear during normal play.

One important note: your current output confirms the basic gameplay, but it doesn't prove that every edge case works. We should test those before considering Task 2 complete.

done

Nice! ✅ Your basic Blackjack gameplay is working.

Now let's finish **Scenario 18 — Task 2** by verifying the remaining edge cases before you submit.

Task 2 checklist

- ◯ Hit and stand work.
- ◯ Dealer reveals cards and determines the winner.
- ◯ Quit works.
- ◯ Play-again prompt works.
- ◯ Natural blackjack.
- ◯ Player bust.
- ◯ Dealer bust.
- ◯ Push (tie).
- ◯ No wager mechanism implemented.

The remaining four cases are difficult to trigger reliably with a shuffled deck. We can test them using controlled hands without changing your `cards.py` or adding dependencies.

Have you already committed and pushed your changes to GitHub, or do you still need to do that?

not yet

Okay 👍 Let's save your Task 2 work to GitHub properly before moving on to Task 3.

Step 1: Check your Git status

In the same PowerShell terminal, run:

```powershell
git status
```

This shows which files you've modified.

Step 2: Review your changes

Run:

```powershell
git diff
```

Check that:

- `game.py` contains the Task 2 changes.
- `cards.py` has not been modified.
- No unnecessary files are included.

Step 3: Stage your changes

Since Task 2 only requires `game.py`, run:

```powershell
git add game.py
```

Step 4: Commit your work

```powershell
git commit -m "Complete Task 2 blackjack game logic"
```

Step 5: Push to GitHub

```powershell
git push
```

If Git reports that the current branch has no upstream, don't guess the branch name. Run `git branch --show-current` and `git status`, then we'll use the correct push command.

Do this now: Start with `git status` and send me the output. I'll guide you through the next step based on your actual repository state.

```
(use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working
directory)
        modified:   cards.py
        modified:   game.py
```

```
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        __pycache__/

no changes added to commit (use "git add" and/or "git commit -
a")
PS C:\Users\manis\Desktop\18_blackjack>
```

Good, we found one important thing before committing. 👍

Your Git status shows:

- `game.py` — modified (Task 2 changes).
- `cards.py` — modified (your earlier Ace-scoring fix).
- `__pycache__/` — untracked Python-generated files.

We should preserve your Ace-scoring fix in `cards.py` without changing it further. Since you want to commit the Task 2 work and the Ace fix is also part of your work, let's first inspect the changes.

Step 1: Check what changed in each file

Run these commands separately in PowerShell:

```powershell
git diff -- cards.py
```

Then:

```powershell
git diff -- game.py
```

Send me both outputs.

Step 2: Ignore generated Python files

The `__pycache__/` directory contains generated Python bytecode. You generally shouldn't commit it.

After reviewing your diffs, we'll add `__pycache__/` to `.gitignore` if your project doesn't already ignore it.

Don't run `git restore cards.py` — that could undo your Ace-scoring fix. We'll keep your changes safe and commit only what belongs in your submission.

```
... hand_value([("A", "S"), ("9", "H")])
... hand_value([("A", "S"), ("9", "H"), ("5", "D")])
... hand_value([("A", "S"), ("A", "H"), ("9", "D")])
... hand_value([("A", "S"), ("A", "H"), ("9", "D"), ("5", "C")])
...
KeyboardInterrupt
>>> from cards import hand_value
>>> hand_value([("A", "S"), ("9", "H")])
20
>>> hand_value([("A", "S"), ("9", "H"), ("5", "D")])
25
>>> exit()
PS C:\Users\manis\Desktop\18_blackjack> python
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC
v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more
information.
>>> from cards import hand_value
>>> hand_value([("A", "S"), ("9", "H"), ("5", "D")])
15
>>> hand_value([("A", "S"), ("A", "H"), ("9", "D")])
21
>>>
... hand_value([("A", "S"), ("9", "H")])
20
>>> hand_value([("A", "S"), ("A", "H"), ("9", "D"), ("5", "C")])
16
>>> exit()
PS C:\Users\manis\Desktop\18_blackjack> python game.py
PS C:\Users\manis\Desktop\18_blackjack> python game.py
Blackjack — starting chips: 100
Dealer: ??
Player: AC 3D = 14
[h]it [s]tand [q]uit: h
Dealer: ??
```

```
Player: AC 3D 10D = 14
[h]it [s]tand [q]uit: s
Dealer: KC 9S
Player: AC 3D 10D = 14
Dealer wins.
Play again? [y/n]: n
PS C:\Users\manis\Desktop\18_blackjack>
>> git status
>>
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working
directory)
        modified:   cards.py
        modified:   game.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        __pycache__/

no changes added to commit (use "git add" and/or "git commit -
a")
PS C:\Users\manis\Desktop\18_blackjack>
>> git diff -- cards.py
>>
diff --git a/cards.py b/cards.py
index 1134aa6..7cdaad3 100644
--- a/cards.py
+++ b/cards.py
@@ -13,8 +13,24 @@ class Deck:
         return self.cards.pop() if self.cards else None



+
+
 def hand_value(hand):
:
```

Good, your Ace-scoring tests are passing, and your Git branch is `main`, up to date with `origin/main`.

Your latest output also reveals something important:

- `hand_value([("A", "S"), ("9", "H")])` returns `20`.
- Adding a 5 changes the total to `15`.
- Two Aces and a 9 return `21`.
- Two Aces, a 9 and a 5 return `16`.

These are all correct. We'll preserve `cards.py` and avoid modifying the Ace-scoring function.

What to do next

Your `git diff -- cards.py` output only shows extra blank lines so far. That doesn't show the full diff, so let's verify whether any other changes exist before committing.

1. Inspect the `game.py` changes

Run:

```powershell
git diff -- game.py
```

Check that it contains your Task 2 changes, including natural blackjack, bust handling, dealer drawing, and pushes.

2. Stage only `game.py`

Since `cards.py` currently appears to have only extra blank lines in its diff, and `__pycache__/` is generated, stage just the game file:

```powershell
git add game.py
git status
```

You should see `game.py` under **Changes to be committed**, while `cards.py` remains unstaged.

3. Commit and push

If the staged diff looks correct, run:

powershell

```
git commit -m "Complete Task 2 blackjack game logic"
git push origin main
```

Don't add `cards.py` or `__pycache__/` to this commit.

First, send me the output of `git diff -- game.py` so we can verify the code before you commit it.

```
PS C:\Users\manis\Desktop\18_blackjack> from cards import
hand_value
At line:1 char:1
+ from cards import hand_value
+ ~~~~
The 'from' keyword is not supported in this version of the
language.
    + CategoryInfo          : ParserError: (:) [],
ParentContainsErrorRecordException
    + FullyQualifiedErrorId : ReservedKeywordNotAllowed

PS C:\Users\manis\Desktop\18_blackjack> python
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC
v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more
information.
>>> from cards import hand_value
>>> hand_value([("A", "S"), ("9", "H"), ("5", "D")])
25
>>> hand_value([("A", "S"), ("A", "H"), ("9", "D")])
31
>>> from cards import hand_value
...
... hand_value([("A", "S"), ("9", "H")])
... hand_value([("A", "S"), ("9", "H"), ("5", "D")])
... hand_value([("A", "S"), ("A", "H"), ("9", "D")])
... hand_value([("A", "S"), ("A", "H"), ("9", "D"), ("5", "C")])
...
KeyboardInterrupt
>>> from cards import hand_value
```

```
>>> hand_value([("A", "S"), ("9", "H")])
20
>>> hand_value([("A", "S"), ("9", "H"), ("5", "D")])
25
>>> exit()
PS C:\Users\manis\Desktop\18_blackjack> python
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC
v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more
information.
>>> from cards import hand_value
>>> hand_value([("A", "S"), ("9", "H"), ("5", "D")])
15
>>> hand_value([("A", "S"), ("A", "H"), ("9", "D")])
21
>>>
... hand_value([("A", "S"), ("9", "H")])
20
>>> hand_value([("A", "S"), ("A", "H"), ("9", "D"), ("5", "C")])
16
>>> exit()
PS C:\Users\manis\Desktop\18_blackjack> python game.py
PS C:\Users\manis\Desktop\18_blackjack> python game.py
Blackjack — starting chips: 100
Dealer: ??
Player: AC 3D = 14
[h]it [s]tand [q]uit: h
Dealer: ??
Player: AC 3D 10D = 14
[h]it [s]tand [q]uit: s
Dealer: KC 9S
Player: AC 3D 10D = 14
Dealer wins.
Play again? [y/n]: n
PS C:\Users\manis\Desktop\18_blackjack>
>> git status
>>
On branch main
Your branch is up to date with 'origin/main'.
```

```
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working
directory)
        modified:   cards.py
        modified:   game.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        __pycache__/

no changes added to commit (use "git add" and/or "git commit -
a")
PS C:\Users\manis\Desktop\18_blackjack>
>> git diff -- cards.py
>>
diff --git a/cards.py b/cards.py
index 1134aa6..7cdaad3 100644
--- a/cards.py
+++ b/cards.py
@@ -13,8 +13,24 @@ class Deck:
         return self.cards.pop() if self.cards else None



+
+
 def hand_value(hand):
:
diff --git a/cards.py b/cards.py
index 1134aa6..7cdaad3 100644
--- a/cards.py
+++ b/cards.py
@@ -13,8 +13,24 @@ class Deck:
         return self.cards.pop() if self.cards else None



+
+
 def hand_value(hand):
:
```

```
diff --git a/cards.py b/cards.py
index 1134aa6..7cdaad3 100644
--- a/cards.py
+++ b/cards.py
@@ -13,8 +13,24 @@ class Deck:
         return self.cards.pop() if self.cards else None



+
+
 def hand_value(hand):
:
diff --git a/cards.py b/cards.py
index 1134aa6..7cdaad3 100644
--- a/cards.py
+++ b/cards.py
@@ -13,8 +13,24 @@ class Deck:
         return self.cards.pop() if self.cards else None



+
+
 def hand_value(hand):
+    value = 0
+    aces = 0
+
+    for rank, _ in hand:
+        if rank == "A":
+            value += 11
+            aces += 1
+        elif rank in {"J", "Q", "K"}:
+            value += 10
+        else:
+            value += int(rank)
+
+    while value > 21 and aces > 0:
+        value -= 10
+        aces -= 1

-    value = sum(11 if rank == "A" else 10 if rank in {"J", "Q", "K"}
```

```
else int(rank)
-               for rank, _ in hand)
    return value
+
diff --git a/cards.py b/cards.py
index 1134aa6..7cdaad3 100644
--- a/cards.py
+++ b/cards.py
@@ -13,8 +13,24 @@ class Deck:
         return self.cards.pop() if self.cards else None



+
+
 def hand_value(hand):
+    value = 0
+    aces = 0
+
+    for rank, _ in hand:
+        if rank == "A":
+            value += 11
+            aces += 1
+        elif rank in {"J", "Q", "K"}:
+            value += 10
+        else:
+            value += int(rank)
+
+    while value > 21 and aces > 0:
+        value -= 10
+        aces -= 1

-    value = sum(11 if rank == "A" else 10 if rank in {"J", "Q", "K"}
else int(rank)
-               for rank, _ in hand)
    return value
+
Invalid mark letter ^I  (press RETURN) not able to paste gam
```

No worries! You're currently stuck inside Git's diff viewer, not PowerShell. That's why you see repeated diffs and the message `Invalid mark letter ^I`.

Let's get you out first.

Step 1: Exit the diff viewer

Press `q` on your keyboard.

You should return to a prompt that looks like:

```powershell
PS C:\Users\manis\Desktop\18_blackjack>
```

Step 2: Don't worry about `cards.py`

Your diff confirms that the Ace-scoring function was changed from a simple sum to logic that correctly adjusts Aces from 11 to 1. That is your working fix, so **don't restore or overwrite it**.

Also, `from cards import hand_value` must be entered inside Python, after running `python`. You cannot enter Python statements directly at the PowerShell prompt.

Step 3: Check the `game.py` diff without opening the viewer

After returning to PowerShell, run:

```powershell
git --no-pager diff -- game.py
```

This prints the changes directly in the terminal, so you can copy the output without getting stuck in the viewer.

Send me that output. Then we'll stage and commit `game.py` while preserving your working `cards.py` fix.

```
                while self.chips > 0:
                    if not self.round():
                        return
+
                    if input("Play again? [y/n]: ").strip().lower() != "y":
                        return
```

```
+
+
+if __name__ == "__main__":
+    Blackjack().run() should record again?
```

Yes, that's correct! 👍 Those lines confirm that your `game.py` includes the code needed to start the game when you run `python game.py`.

</> Python

```python
if __name__ == "__main__":
    Blackjack().run()
```

You **don't need to record or re-enter that code again** if it's already saved in `game.py`.

Now save your Task 2 changes to GitHub

Run these commands in PowerShell, one at a time:

1. Stage only `game.py`

</> powershell

```
git add game.py
```

2. Check what's staged

</> powershell

```
git status
```

Make sure `game.py` appears under **Changes to be committed**. Your `cards.py` fix should remain unstaged for now.

3. Commit the changes

</> powershell

```
git commit -m "Complete Task 2 blackjack game logic"
```

4. Push to GitHub

```powershell
git push origin main
```

After that, your Task 2 changes should be on GitHub.

Important: If your lab requires the Ace-scoring fix in `cards.py` to be submitted too, we'll need to commit that fix separately. Don't discard it.

```
+
+
+if __name__ == "__main__":
+    Blackjack().run()
PS C:\Users\manis\Desktop\18_blackjack> git add game.py
PS C:\Users\manis\Desktop\18_blackjack> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   game.py

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working
directory)
        modified:   cards.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        __pycache__/

PS C:\Users\manis\Desktop\18_blackjack
```

Perfect! ✅ Your Git status is exactly what we want.

- `game.py` — staged and ready to commit.
- `cards.py` — your Ace-scoring fix is preserved and unstaged.
- `__pycache__/` — generated files, which we won't commit.

Next step: Commit Task 2

Run this command:

powershell

```
git commit -m "Complete Task 2 blackjack game logic"
```

Then push it to GitHub:

powershell

```
git push origin main
```

After both commands finish, send me the terminal output.

We'll then decide whether your `cards.py` fix also needs its own commit for the lab submission.

```
54 insertions(+), 8 deletions(-)
PS C:\Users\manis\Desktop\18_blackjack> git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 20 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 852 bytes | 852.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local
objects.
To https://github.com/manishribil/18_blackjack.git
   334f58a..1ee8af3  main -> main
PS C:\Users\manis\Desktop\18_blackjack>
```

Excellent! 🎉 **Task 2 has been committed and pushed to GitHub successfully.**

Your push output confirms that `main` was updated from `334f58a` to `1ee8af3`.

Repository: 18_blackjack on GitHub ↗

Current status

`game.py` — Done
Task 2 committed and pushed

`cards.py` — Pending
Ace-scoring fix preserved locally; not yet committed

Edge-case testing
Natural blackjack, busts and pushes still need dedicated verification.

Next step

Before starting Task 3, let's make sure your Ace-scoring fix is also saved if it is required for your lab submission.

Run:

powershell

```
git add cards.py
git commit -m "Fix Ace scoring in blackjack"
git push origin main
```

This creates a separate commit for `cards.py` without changing your completed Task 2 commit.

Once that's done, we'll verify the remaining Task 2 edge cases and then move to Task 3 — the wager mechanism.

umerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 20 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 584 bytes | 584.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/manishribil/18_blackjack.git

```
1ee8af3..6969083  main -> main
PS C:\Users\manis\Desktop\18_blackjack>
```

Excellent! 🎉 Both commits are now pushed to GitHub.

Your GitHub progress

✅	**Task 2 — Blackjack game logic** Natural blackjack, bust handling, dealer draws, pushes and round termination.	Pushed
✅	**Ace-scoring fix —** `cards.py` Saved in a separate commit.	Pushed

Repository: View 18_blackjack on GitHub ↗

One final verification

Run these commands:

powershell

```
git status
```

powershell

```
git log -2 --oneline
```

Your branch should be up to date with `origin/main`, and the latest two commits should appear in the log.

Important: The code is pushed, but we haven't yet independently verified all the edge cases (natural blackjack, player bust, dealer bust and push). Let's test those before marking Task 2 fully verified.

```
nch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        __pycache__/
```

nothing added to commit but untracked files present (use "git add" to track)
PS C:\Users\manis\Desktop\18_blackjack> git log -2 --oneline
6969083 (HEAD -> main, origin/main, origin/HEAD) Fix Ace scoring in blackjack
1ee8af3 Complete Task 2 blackjack game logic
PS C:\Users\manis\Desktop\18_blackjack>

Perfect! 🎉 Your GitHub repository is up to date, and both commits are present.

- `1ee8af3` — Complete Task 2 blackjack game logic
- `6969083` — Fix Ace scoring in blackjack

Your only remaining Git status item is `__pycache__/`, which contains generated Python bytecode. It does not need to be committed.

Clean up the repository

To stop Git from showing `__pycache__/` as untracked, create or update `.gitignore`.

Run:

```powershell
Add-Content .gitignore "`n__pycache__/"
```

Then commit and push the ignore rule:

```powershell
git add .gitignore
git commit -m "Ignore Python cache files"
git push origin main
```

If `.gitignore` doesn't exist, PowerShell will create it.

What's next?

Your code is pushed, but Task 2 still needs dedicated edge-case testing for natural blackjack, player bust, dealer bust, and pushes.

I recommend we test those cases before moving on to Task 3.

.gitignore
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 20 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 298 bytes | 298.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/manishribil/18_blackjack.git
  6969083..6b05fbd main -> main
PS C:\Users\manis\Desktop\18_blackjack>

done

\Desktop\18_blackjack> git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
PS C:\Users\manis\Desktop\18_blackjack>

test3?

ig test2 is done we need to proceed to next

I have completed Task 2 of Scenario 18 — Blackjack vs Dealer.
Now implement Task 3: Multi-round Bankroll.
Inspect my current game.py and cards.py before making changes.
Requirements:

1. Keep the bankroll in memory across multiple rounds, starting at 100 chips.
2. Ask the player to enter a wager before each round.
3. Validate the wager: it must be a valid positive integer and must not exceed the available bankroll.
4. Invalid wagers must not change the bankroll or start a round.

5. A win should increase the bankroll by the wager amount, a loss should decrease it by the wager amount, and a push should leave it unchanged.
6. Ensure each round's result is settled exactly once, including natural blackjack, player bust, dealer bust and push outcomes.
7. Prevent duplicate settlement if the round ends or a command is repeated.
8. Keep the existing terminal-based game and project structure. Do not add persistent storage or unnecessary dependencies.
9. Preserve my completed Task 1 and Task 2 behavior.
10. Explain every change and provide test cases for wins, losses, pushes, invalid wagers, repeated commands and multiple rounds.

First explain the proposed changes. Then provide the updated code for the necessary files.
from cards import Deck, hand_value

```
class Blackjack:
    def __init__(self):
        self.chips = 100

    def show(self, player, dealer, hide=True):
        shown_dealer = (
            ["??"] if hide
            else [f"{r}{s}" for r, s in dealer]
        )
        print("Dealer:", " ".join(shown_dealer))
        print(
            "Player:", " ".join(f"{r}{s}" for r, s in player),
            "=", hand_value(player)
        )

    def round(self):
        deck = Deck()
        player = [deck.draw(), deck.draw()]
        dealer = [deck.draw(), deck.draw()]

        self.show(player, dealer)
```

```
pv = hand_value(player)
dv = hand_value(dealer)

# Task 2: Check natural blackjack before player actions.
player_natural = pv == 21 and len(player) == 2
dealer_natural = dv == 21 and len(dealer) == 2

if player_natural or dealer_natural:
    self.show(player, dealer, hide=False)

    if player_natural and dealer_natural:
        print("Push. Both have blackjack.")
    elif player_natural:
        print("Player wins with blackjack.")
    else:
        print("Dealer wins with blackjack.")

    return True

# Player's turn
while hand_value(player) < 21:
    key = input("[h]it [s]tand [q]uit: ").strip().lower()

    if key == "q":
        return False

    if key == "s":
        break

    if key == "h":
        player.append(deck.draw())
        self.show(player, dealer)

        if hand_value(player) > 21:
            print("Bust. Player loses.")
            self.show(player, dealer, hide=False)
            return True

# Dealer draws until reaching at least 17.
```

```
        while hand_value(dealer) < 17:
            dealer.append(deck.draw())

        # Reveal both hands and resolve the round.
        self.show(player, dealer, hide=False)

        pv = hand_value(player)
        dv = hand_value(dealer)

        if dv > 21:
            print("Dealer busts. Player wins.")
        elif pv > dv:
            print("Player wins.")
        elif pv < dv:
            print("Dealer wins.")
        else:
            print("Push.")

        # Task 2: No chip changes or wager mechanism yet.
        return True

    def run(self):
        print("Blackjack — starting chips:", self.chips)

        while self.chips > 0:
            if not self.round():
                return

            if input("Play again? [y/n]: ").strip().lower() != "y":
                return


if __name__ == "__main__":
    Blackjack().run()
```

game.py

Worked for 21s

```
anis\Desktop\18_blackjack> python game.py
Blackjack — starting chips: 100
```

Enter wager (1-100) or q to quit:

C:\Users\manis\Desktop\18_blackjack> python game.py
Blackjack — starting chips: 100
Enter wager (1-100) or q to quit: 0
Invalid wager. Wager must be greater than 0.
Enter wager (1-100) or q to quit: 12
Dealer: ??
Player: 9D 7C = 16
[h]it [s]tand [q]uit: -5
Invalid choice. Enter h, s, or q.
[h]it [s]tand [q]uit: 70
Invalid choice. Enter h, s, or q.
[h]it [s]tand [q]uit:

Invalid choice. Enter h, s, or q.
[h]it [s]tand [q]uit: s
Dealer: AS 4D 10C 4H
Player: 9D 7C = 16
Dealer wins.
Bankroll: 88 chips
Play again? [y/n]:

Dealer wins.
Bankroll: 88 chips
Play again? [y/n]: y
Enter wager (1-88) or q to quit: abc
Invalid wager. Enter a positive whole number.
Enter wager (1-88) or q to quit: 2.5
Invalid wager. Enter a positive whole number.
Enter wager (1-88) or q to quit: -5
Invalid wager. Wager must be greater than 0.
Enter wager (1-88) or q to quit: 150
Invalid wager. Not enough chips.
Enter wager (1-88) or q to quit:

Enter wager (1-88) or q to quit: 20
Dealer: ??
Player: 5D KS = 15

```
[h]it [s]tand [q]uit: h
Dealer: ??
Player: 5D KS 8D = 23
Bust. Player loses.
Dealer: 10D 9C
Player: 5D KS 8D = 23
Bankroll: 68 chips
Play again? [y/n]:
```

```
Desktop\18_blackjack> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working
directory)
        modified:   game.py

no changes added to commit (use "git add" and/or "git commit -
a")
PS C:\Users\manis\Desktop\18_blackjack>
```

```
S C:\Users\manis\Desktop\18_blackjack> git add game.py
>> git commit -m "Implement blackjack multi-round bankroll"
>> git push origin main
[main 3775c46] Implement blackjack multi-round bankroll
 1 file changed, 82 insertions(+), 11 deletions(-)
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 20 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 1.24 KiB | 1.24 MiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local
objects.
To https://github.com/manishribil/18_blackjack.git
   6b05fbd..3775c46  main -> main
PS C:\Users\manis\Desktop\18_blackjack>
```

```
S C:\Users\manis\Desktop\18_blackjack> git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
PS C:\Users\manis\Desktop\18_blackjack>
```

```
>>> from game import Blackjack
...
... game = Blackjack()
... game._settle_wager(20, "win", 1)
... print(game.chips)
... game._settle_wager(20, "win", 1)
... print(game.chips)
...
```

```
Bankroll: 120 chips
120
Round already settled. Ignoring duplicate settlement.
120
>>>
```

```
nkroll: 100 chips
100
>>>
```

```
Bankroll: 120 chips
120
>>>
```

task3 done/

I have completed Tasks 1, 2 and 3 of Scenario 18 — Blackjack vs Dealer.
Now implement Task 4: Action Feedback and Robustness.
Inspect my current game.py and cards.py before making changes.
Requirements:

1. Display concise feedback whenever a card is actually drawn.
2. Display clear round outcome feedback: player win, dealer win, push, player bust and natural blackjack where applicable.
3. Handle invalid player commands without crashing or changing the bankroll.
4. Reject invalid wagers without changing the bankroll.
5. Handle an empty or depleted deck safely. Do not attempt to score or display None as a card.
6. Handle quitting cleanly at the appropriate prompts.
7. Preserve the existing Ace scoring, round resolution, wager validation and exactly-once settlement logic.
8. Keep all state in memory. Do not add CSV, JSON, SQLite or other persistence.
9. Do not add unnecessary external dependencies or replace the entire project.
10. Explain every change and provide test cases with expected results.

First explain the proposed changes, then provide the minimal code changes needed.
from cards import Deck, hand_value

```
class Blackjack:
    def __init__(self):
        self.chips = 100
        self._round_counter = 0
        self._settled_rounds = set()

    def show(self, player, dealer, hide=True):
        shown_dealer = (
            ["??"] if hide
            else [f"{r}{s}" for r, s in dealer]
        )
        print("Dealer:", " ".join(shown_dealer))
        print(
            "Player:", " ".join(f"{r}{s}" for r, s in player),
            "=", hand_value(player)
        )
```

```
    def round(self):
        deck = Deck()
        player = [deck.draw(), deck.draw()]
        dealer = [deck.draw(), deck.draw()]

        self.show(player, dealer)

        pv = hand_value(player)
        dv = hand_value(dealer)

        # Task 2: Check natural blackjack.
        player_natural = pv == 21 and len(player) == 2
        dealer_natural = dv == 21 and len(dealer) == 2

        if player_natural or dealer_natural:
            self.show(player, dealer, hide=False)

            if player_natural and dealer_natural:
                print("Push. Both have blackjack.")
                return "push"
            elif player_natural:
                print("Player wins with blackjack.")
                return "win"
            else:
                print("Dealer wins with blackjack.")
                return "loss"

        # Player's turn.
        while hand_value(player) < 21:
            key = input("[h]it [s]tand [q]uit: ").strip().lower()

            if key == "q":
                print("Round cancelled. Wager returned.")
                return "cancelled"

            elif key == "s":
                break

            elif key == "h":
                player.append(deck.draw())
```

```
                self.show(player, dealer)

                if hand_value(player) > 21:
                    print("Bust. Player loses.")
                    self.show(player, dealer, hide=False)
                    return "loss"

            else:
                print("Invalid choice. Enter h, s, or q.")

        # Dealer draws until reaching at least 17.
        while hand_value(dealer) < 17:
            dealer.append(deck.draw())

        # Reveal both hands and resolve the outcome.
        self.show(player, dealer, hide=False)

        pv = hand_value(player)
        dv = hand_value(dealer)

        if dv > 21:
            print("Dealer busts. Player wins.")
            return "win"
        elif pv > dv:
            print("Player wins.")
            return "win"
        elif pv < dv:
            print("Dealer wins.")
            return "loss"
        else:
            print("Push.")
            return "push"

    def _settle_wager(self, wager, outcome, round_id):
        # Guard against settling the same round more than once.
        if round_id in self._settled_rounds:
            print("Round already settled. Ignoring duplicate
settlement.")
            return False
```

```
        # Mark the round before changing the bankroll.
        self._settled_rounds.add(round_id)

        if outcome == "win":
            self.chips += wager
        elif outcome == "loss":
            self.chips -= wager
        elif outcome in ("push", "cancelled"):
            pass
        else:
            # Do not accept unknown outcomes.
            self._settled_rounds.remove(round_id)
            raise ValueError(f"Unknown round outcome: {outcome}")

        if outcome == "cancelled":
            print("No chips lost. Wager returned.")

        print("Bankroll:", self.chips, "chips")
        return True

    def run(self):
        print("Blackjack — starting chips:", self.chips)

        while self.chips > 0:
            # Collect and validate a wager before dealing cards.
            while True:
                raw_wager = input(
                    f"Enter wager (1-{self.chips}) or q to quit: "
                ).strip().lower()

                if raw_wager == "q":
                    return

                try:
                    wager = int(raw_wager)
                except ValueError:
                    print("Invalid wager. Enter a positive whole number.")
                    continue

                if wager <= 0:
```

```
                print("Invalid wager. Wager must be greater than 0.")
                continue

            if wager > self.chips:
                print("Invalid wager. Not enough chips.")
                continue

            break

        self._round_counter += 1
        round_id = self._round_counter

        outcome = self.round()

        # Every started round is settled at most once.
        self._settle_wager(wager, outcome, round_id)

        if outcome == "cancelled":
            return

        if self.chips <= 0:
            print("Game over. You have no chips left.")
            return

        if input("Play again? [y/n]: ").strip().lower() != "y":
            return


if __name__ == "__main__":
    Blackjack().run()
game.py    import random

RANKS = ["2", "3", "4", "5", "6", "7", "8", "9", "10", "J", "Q", "K", "A"]
SUITS = ["C", "D", "H", "S"]


class Deck:
    def __init__(self):
        self.cards = [(rank, suit) for suit in SUITS for rank in RANKS]
        random.shuffle(self.cards)
```

```
    def draw(self):
        return self.cards.pop() if self.cards else None




def hand_value(hand):
    value = 0
    aces = 0

    for rank, _ in hand:
        if rank == "A":
            value += 11
            aces += 1
        elif rank in {"J", "Q", "K"}:
            value += 10
        else:
            value += int(rank)

    while value > 21 and aces > 0:
        value -= 10
        aces -= 1

    return value
```

card.py

```
PS C:\Users\manis\Desktop\18_blackjack> python game.py
Blackjack — starting chips: 100
Enter wager (1-100) or q to quit:
```

```
C:\Users\manis\Desktop\18_blackjack> python game.py
Blackjack — starting chips: 100
Enter wager (1-100) or q to quit: abc
Invalid wager. Enter a positive whole number.
Enter wager (1-100) or q to quit: 0
Invalid wager. Wager must be greater than 0.
Enter wager (1-100) or q to quit: -5
```

Invalid wager. Wager must be greater than 0.
Enter wager (1-100) or q to quit: 150
Invalid wager. Not enough chips.
Enter wager (1-100) or q to quit:

id wager. Not enough chips.
Enter wager (1-100) or q to quit: 20
Player draws: 6C
Player draws: 4C
Dealer draws: 6S
Dealer receives a face-down card.
Dealer: ??
Player: 6C 4C = 10
[h]it [s]tand [q]uit:

PS C:\Users\manis\Desktop\18_blackjack> python game.py
Blackjack — starting chips: 100
Enter wager (1-100) or q to quit: x
Invalid wager. Enter a positive whole number.
Enter wager (1-100) or q to quit:

draws: 8H
Dealer receives a face-down card.
Dealer: ??
Player: 9H 7D = 16
[h]it [s]tand [q]uit: x
Invalid choice. Enter h, s

Dealer draws: 8H
Dealer receives a face-down card.
Dealer: ??
Player: 9H 7D = 16
[h]it [s]tand [q]uit: x
Invalid choice. Enter h, s, or q.
[h]it [s]tand [q]uit: q
Round cancelled. Wager returned.
No chips lost. Wager returned.

```
        Bankroll: 100 chips
        PS C:\Users\manis\Desktop\18_blackjack>


...
Deck is empty. Round cancelled; wager returned.
cancelled
>>>
>>>
```

```
ers\manis\Desktop\18_blackjack> git diff --check
>> git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working
directory)
        modified:   game.py

no changes added to commit (use "git add" and/or "git commit -
a")
PS C:\Users\manis\Desktop\18_blackjack>


insertions(+), 20 deletions(-)
PS C:\Users\manis\Desktop\18_blackjack> git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 20 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 1.13 KiB | 1.13 MiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local
objects.
To https://github.com/manishribil/18_blackjack.git
   3775c46..9d21d55  main -> main
PS C:\Users\manis\Desktop\18_blackjack>
```

rs\manis\Desktop\18_blackjack> git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
PS C:\Users\manis\Desktop\18_blackjack>